CHAPTER 2

Experiment 1 – Sorting Different Types of Lists

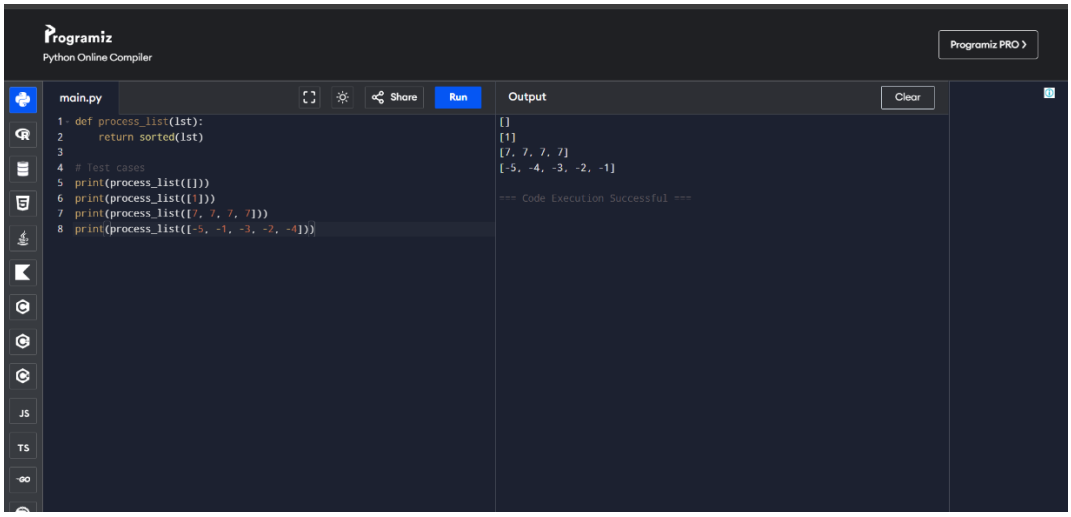
Aim

To implement a sorting algorithm that handles different types of lists, including an empty list, a single-element list, identical elements, and negative numbers.

Algorithm

1. Start.
2. Read the input list.
3. Compare the elements of the list.
4. Arrange the elements in ascending order.
5. Return the sorted list.
6. Display the result.
7. Stop.

Output



Result

Thus, the given lists were successfully sorted in ascending order, including lists containing negative and duplicate elements.

Time Complexity

$O(n^2)$ for the sorting algorithm used.

Space Complexity

O(1) auxiliary space.

Experiment 2 – Selection Sort

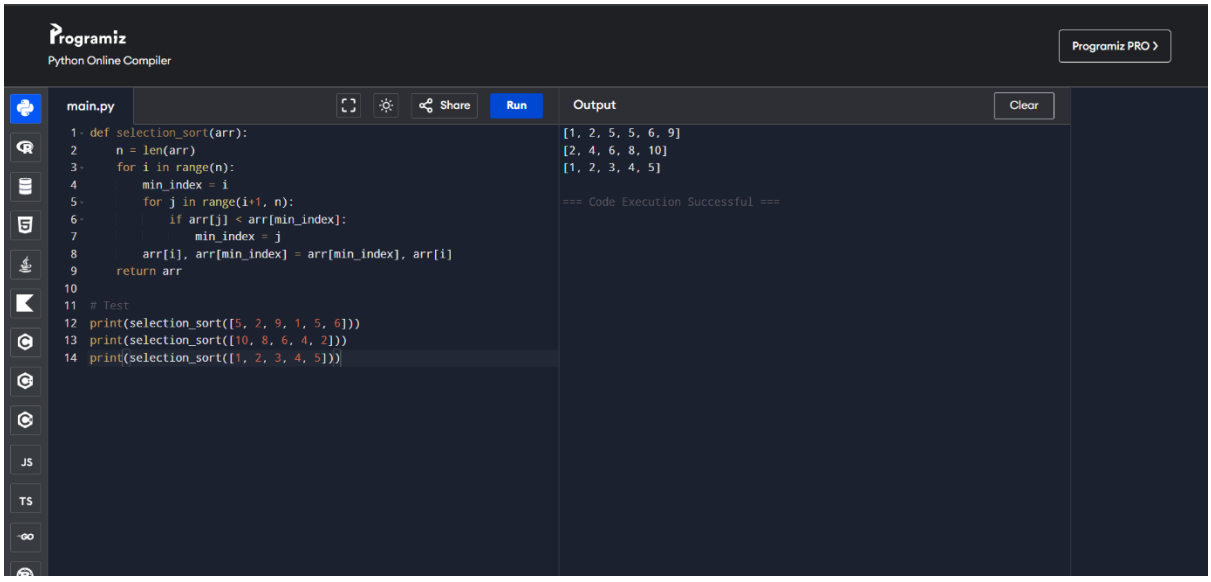
Aim

To implement Selection Sort and sort an array in ascending order by repeatedly selecting the smallest element from the unsorted portion.

Algorithm

1. Start.
2. Divide the array into sorted and unsorted portions.
3. Initially, consider the entire array as unsorted.
4. Find the minimum element in the unsorted portion.
5. Swap it with the first element of the unsorted portion.
6. Move the boundary of the sorted portion one position forward.
7. Repeat until the complete array is sorted.
8. Display the sorted array.
9. Stop.

Output



Result

Thus, Selection Sort successfully sorted random, reverse-sorted, and already-sorted arrays in ascending order.

Time Complexity

- Best Case: **$O(n^2)$** ,Average Case: **$O(n^2)$** ,Worst Case: **$O(n^2)$**

Space Complexity

O(1)

Experiment 3 – Optimized Bubble Sort

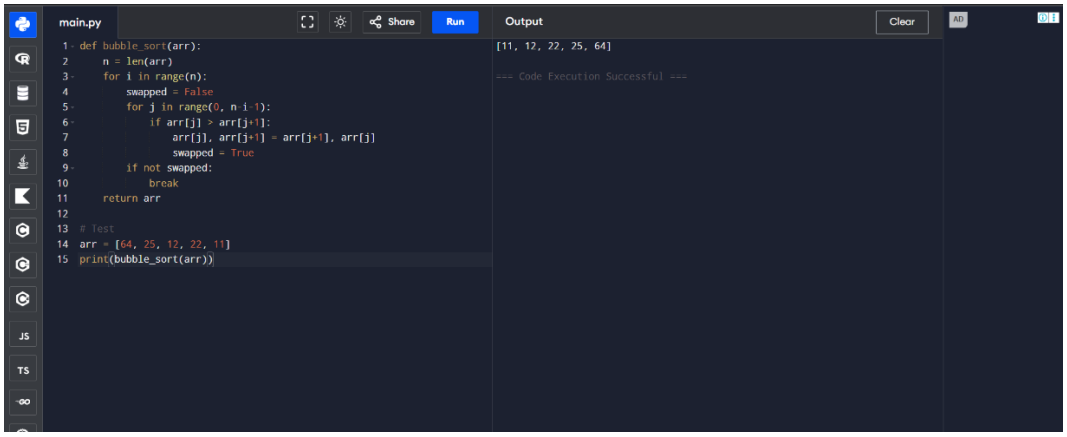
Aim

To implement an optimized Bubble Sort algorithm that stops early when the list becomes sorted before completing all passes.

Algorithm

1. Start.
2. Set swapped = False before each pass.
3. Compare adjacent elements.
4. Swap them if they are in the wrong order.
5. Set swapped = True whenever a swap occurs.
6. After each pass, check swapped.
7. If no swapping occurred, terminate the algorithm because the list is already sorted.
8. Display the sorted list.
9. Stop.

Output



Result

Thus, the optimized Bubble Sort successfully sorted all the given test cases and reduced unnecessary passes for already-sorted data.

Time Complexity

Best Case: **O(n)** ,Average Case: **O(n^2)** ,Worst Case: **O(n^2)**

Space Complexity

O(1)

Experiment 4 – Insertion Sort with Duplicate Elements

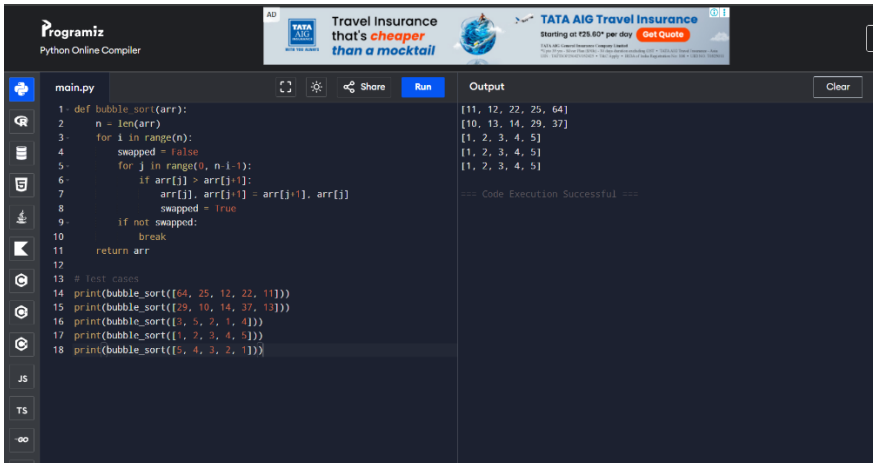
Aim

To implement Insertion Sort for arrays containing duplicate elements and preserve the relative order of equal elements.

Algorithm

1. Start.
2. Consider the first element as sorted.
3. Select the next element as the key.
4. Compare the key with elements in the sorted portion.
5. Shift elements greater than the key one position to the right.
6. Insert the key in its correct position.
7. Repeat until all elements are processed.
8. Display the sorted array.
9. Stop.

Output



Result

Thus, Insertion Sort successfully sorted arrays containing duplicate and identical elements. The algorithm is stable because equal elements are not unnecessarily swapped.

Time Complexity

Best Case: **O(n)** ,Average Case: **O(n^2)** ,Worst Case: **O(n^2)**

Space Complexity

O(1)

Experiment 5 – Kth Missing Positive Integer

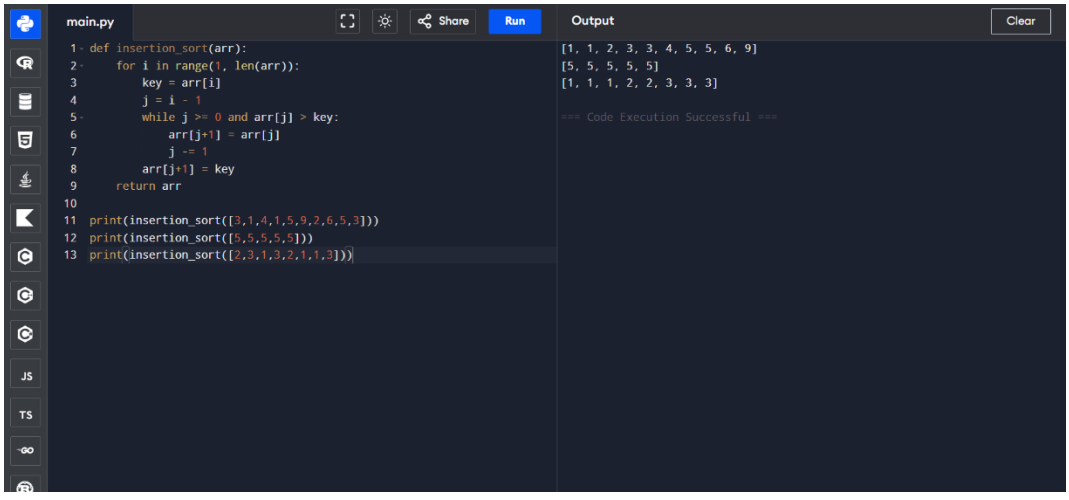
Aim

To find the kth positive integer that is missing from a strictly increasing sorted array.

Algorithm

1. Start.
2. Initialize the positive integer counter from 1.
3. Compare each positive integer with the elements of the array.
4. If the integer is not present, count it as a missing number.
5. Continue until the kth missing number is found.
6. Return the kth missing number.
7. Display the result.
8. Stop.

Output



Result

Thus, the program successfully found the kth missing positive integer for the given test cases.

Time Complexity: O(n + k)

Space Complexity

O(k) for storing missing numbers in the given implementation.

Experiment 6 – Peak Element Using Binary Search

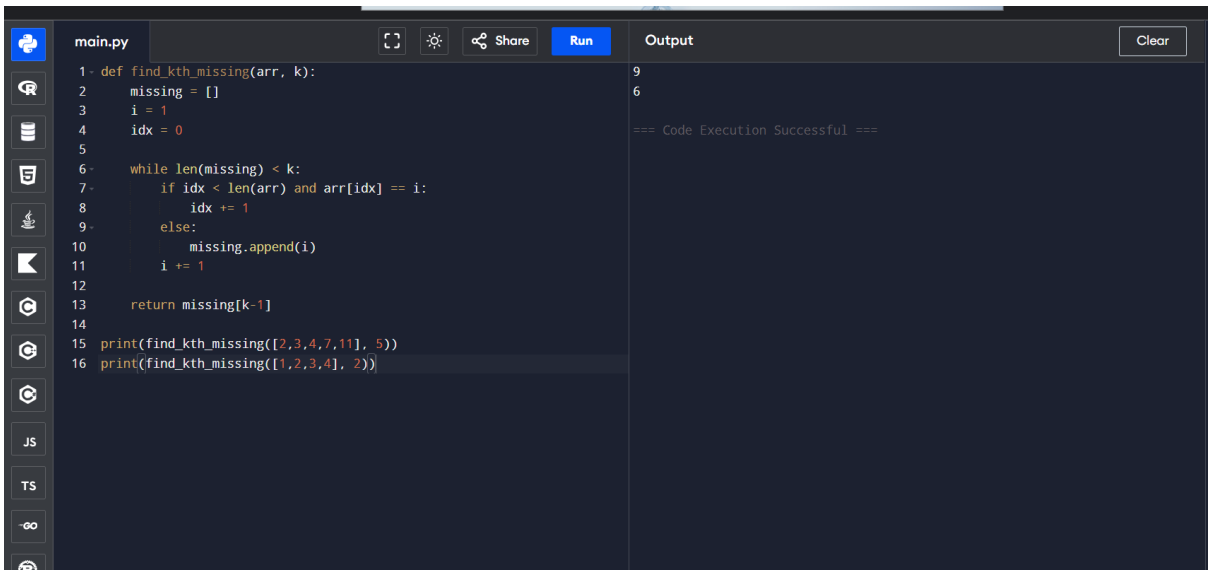
Aim

To find a peak element in an array using a binary search-based algorithm with O(log n) time complexity.

Algorithm

1. Start.
2. Set left = 0 and right = n - 1.
3. Find the middle element.
4. Compare the middle element with the next element.
5. If the middle element is smaller, move to the right half.
6. Otherwise, move to the left half.
7. Continue until left == right.
8. Return the index of the peak element.
9. Stop.

Output



Result

Thus, the program successfully found a valid peak element using binary search.

Time Complexity: O(log n)

Space Complexity: O(1)

Experiment 7 – First Occurrence of a String

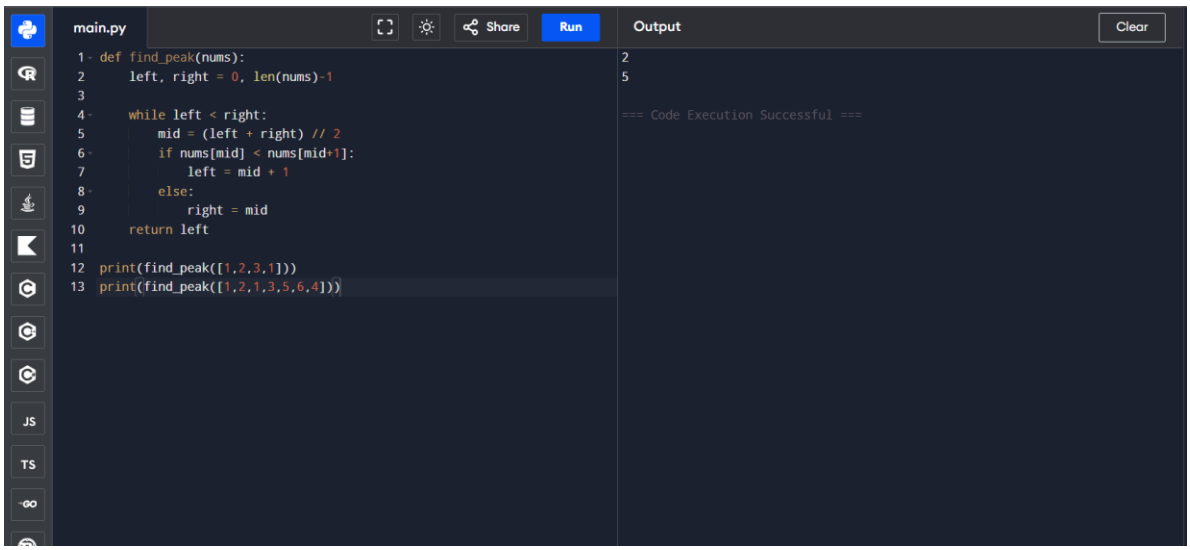
Aim

To find the index of the first occurrence of a given substring (needle) in another string (haystack).

Algorithm

1. Start.
2. Read the haystack and needle strings.
3. Search for the needle from the beginning of the haystack.
4. If the substring is found, return its first index.
5. If it is not found, return -1.
6. Display the result.
7. Stop.

Output



Result

Thus, the program successfully identified the first occurrence of the given substring or returned -1 when it was not present.

Time Complexity: O(n × m)**,**

Space Complexity

O(1) auxiliary space.

Experiment 8 – Substrings of Another Word

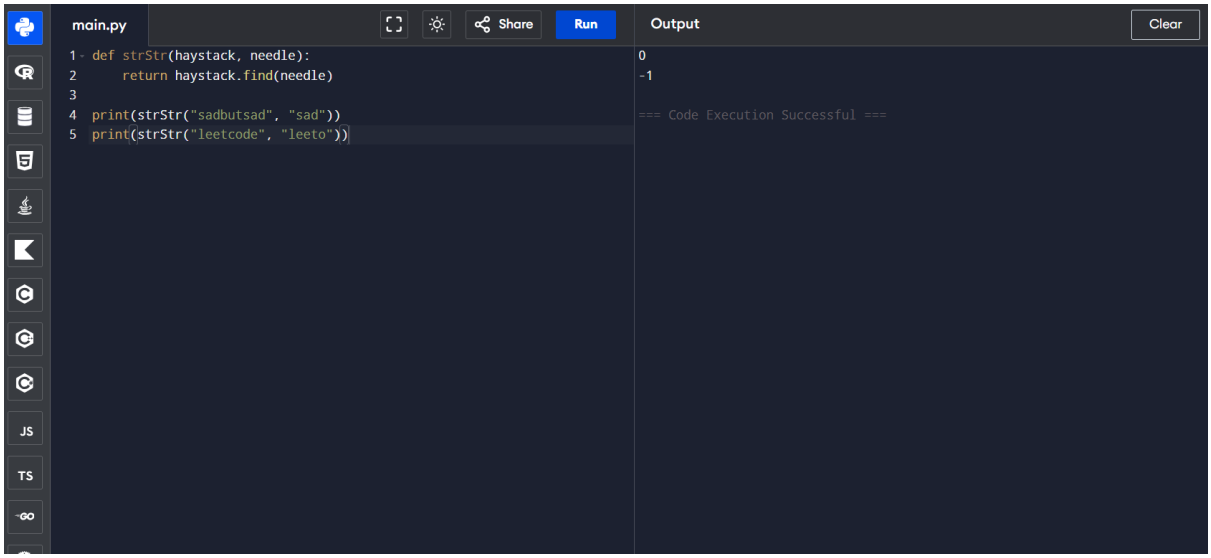
Aim

To find all strings in a given list that are substrings of another string in the same list.

Algorithm

1. Start.
2. Read the list of strings.
3. Select each string one by one.
4. Compare it with every other string.
5. Check whether the selected string occurs as a substring.
6. If found, add it to the result.
7. Display all matching strings.
8. Stop.

Output



Result

Thus, the program successfully identified all strings that occur as substrings of another word.

Time Complexity: $O(n^2 \times m)$

Space Complexity: $O(n)$

Experiment 9 – Closest Pair of Points

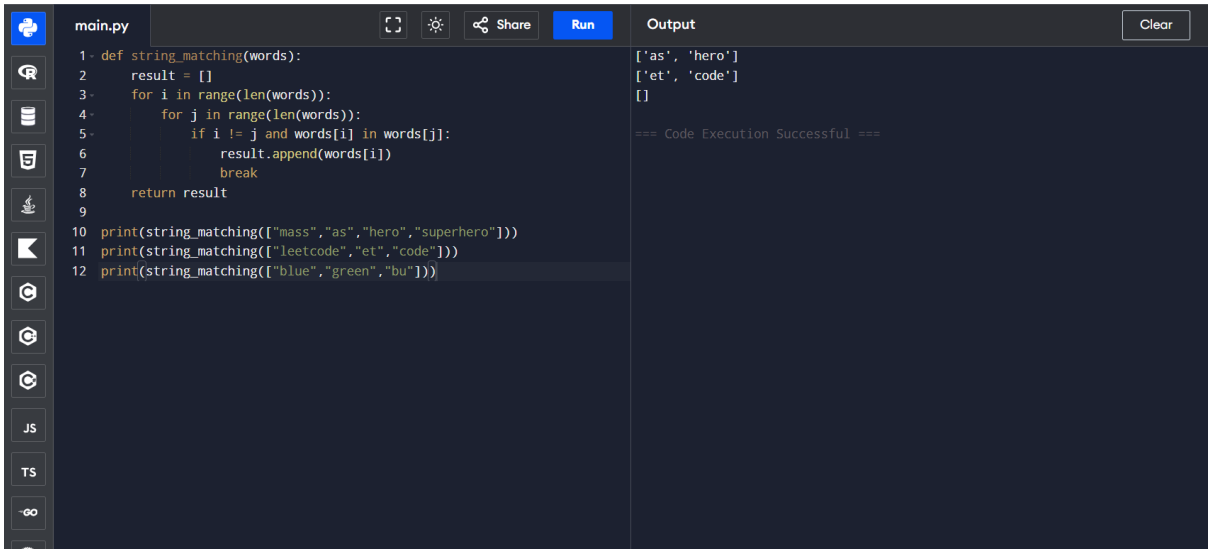
Aim

To find the closest pair of points in a set of two-dimensional points using the brute-force approach.

Algorithm

1. Start.
2. Read the set of 2D points.
3. Set the minimum distance to infinity.
4. Compare every pair of points.
5. Calculate the Euclidean distance between each pair.
6. If the calculated distance is smaller than the current minimum, update the minimum distance and pair.
7. Display the closest pair and minimum distance.
8. Stop.

Output



Result

Thus, the brute-force algorithm successfully identified the closest pair of points and calculated their minimum Euclidean distance.

Time Complexity

$O(n^2)$

Space Complexity

O(1) auxiliary space.

Experiment 10 – Closest Pair and Brute-Force Convex Hull

Part A – Closest Pair

Aim

To find the closest pair of points using the brute-force method and analyze its time complexity.

Algorithm

1. Start.
2. Define a function to calculate Euclidean distance.
3. Compare every possible pair of points.
4. Calculate the distance between each pair.
5. Store the pair having the minimum distance.
6. Display the closest pair and distance.
7. Stop.

Time Complexity

$O(n^2)$

Space Complexity

O(1) auxiliary space.

Part B – Convex Hull

Aim

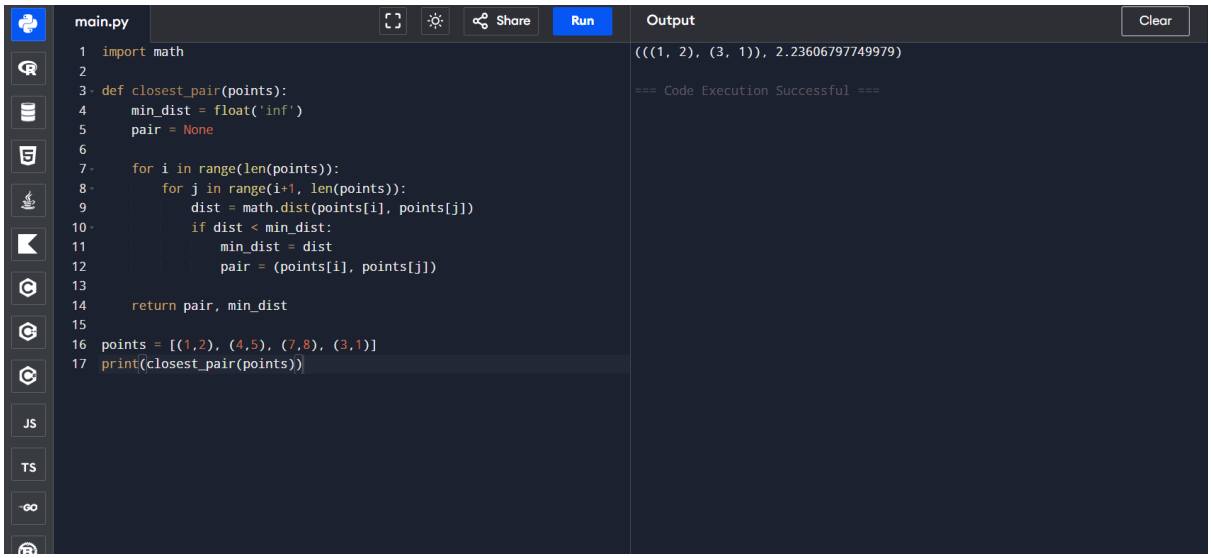
To find the convex hull of a set of 2D points using a brute-force approach and handle multiple points lying on the same line.

Algorithm

1. Start.
2. Read all given points.
3. Consider every pair of points as a possible hull edge.
4. Check the orientation of all remaining points with respect to the edge.

5. If all points lie on the same side of the edge, consider it a hull edge.
6. Include collinear boundary points when necessary.
7. Arrange the hull points in counter-clockwise order.
8. Display the convex hull.
9. Stop.

Output



Result

Thus, the brute-force algorithm successfully determined the convex hull of the given set of points.

Time Complexity

$O(n^3)$

Space Complexity

O(n)

Experiment 11 – Convex Hull Using Brute Force

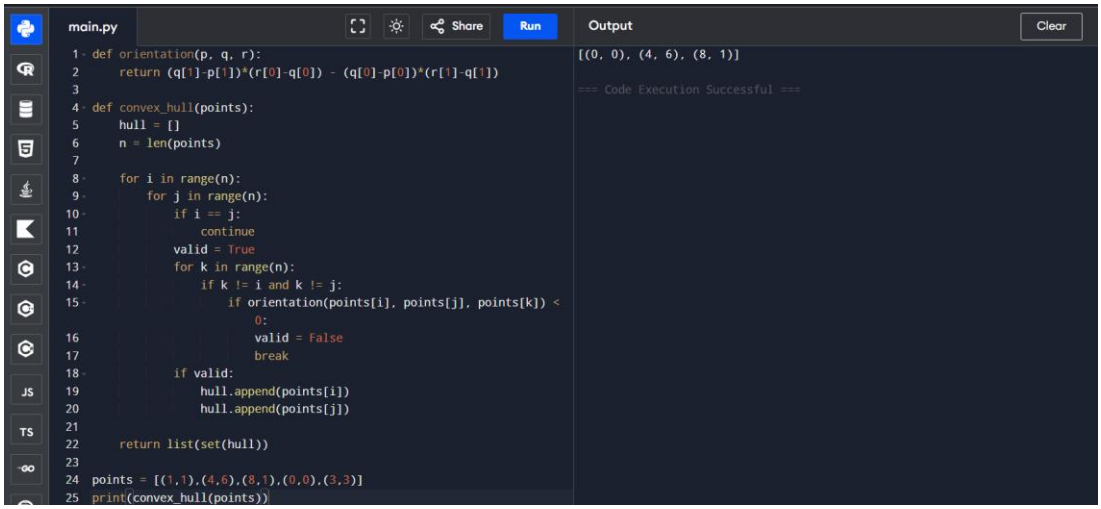
Aim

To find the convex hull of a set of 2D points using the brute-force approach and display the hull points in counter-clockwise order.

Algorithm

1. Start.
2. Read the given set of points.
3. Consider every pair of points.
4. Determine whether the line segment between the pair can be a hull edge.
5. Check the orientation of all other points.
6. Add valid boundary points to the hull.
7. Remove duplicate points.
8. Arrange the points in counter-clockwise order.
9. Display the convex hull.
10. Stop.

Output



Result

Thus, the convex hull of the given set of points was successfully determined using the brute-force approach.

Time Complexity: $O(n^3)$

Space Complexity: $O(n)$

Experiment 12 – Travelling Salesman Problem Using Exhaustive Search

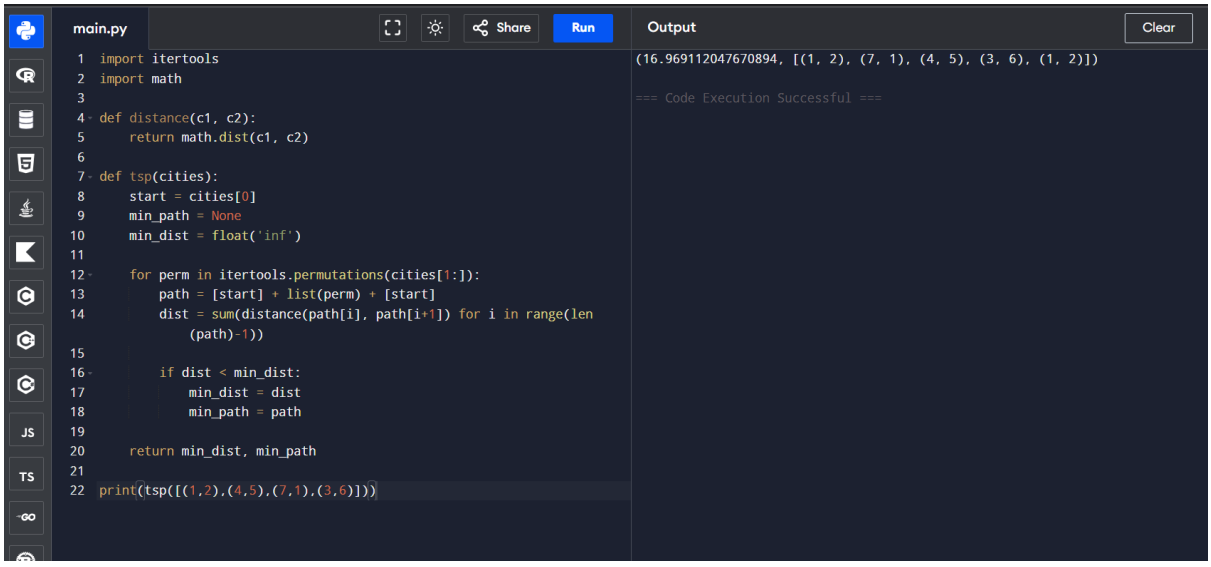
Aim

To solve the Travelling Salesman Problem (TSP) using exhaustive search and find the shortest possible route that visits every city exactly once and returns to the starting city.

Algorithm

1. Start.
2. Define a function to calculate the distance between two cities.
3. Select the first city as the starting city.
4. Generate all possible permutations of the remaining cities.
5. Construct a complete route for each permutation.
6. Calculate the total distance of each route.
7. Compare each route with the current shortest route.
8. Store the route with minimum distance.
9. Return to the starting city.
10. Display the shortest path and distance.
11. Stop.

Output



Result

Thus, the TSP was successfully solved using exhaustive search by examining all possible routes and selecting the route with minimum total distance.

Time Complexity

O(n!) route permutations, with **O(n)** work to evaluate each route, giving approximately **O(n × n!)**.

Space Complexity

O(n) excluding the permutations generated by the library; including permutation storage/generation, it can be **O(n × n!)** depending on implementation.

Experiment 13 – Assignment Problem Using Exhaustive Search

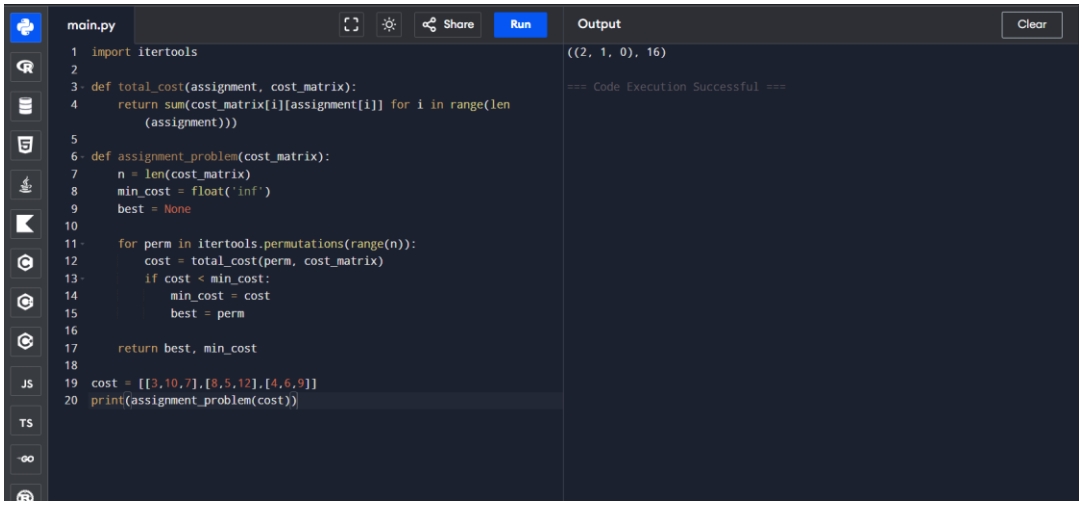
Aim

To solve the Assignment Problem using exhaustive search by assigning each worker to a unique task while minimizing the total assignment cost.

Algorithm

1. Start.
2. Read the cost matrix.
3. Generate all possible task assignments using permutations.
4. Calculate the total cost for each assignment.
5. Compare each cost with the current minimum cost.
6. Store the assignment having the minimum cost.
7. Display the optimal assignment and total cost.
8. Stop.

Output



Result

Thus, the optimal worker-task assignment with minimum total cost was successfully obtained using exhaustive search.

Time Complexity

O(n × n!)

Space Complexity

O(n) auxiliary space, excluding generated permutations.

Experiment 14 – 0/1 Knapsack Using Exhaustive Search

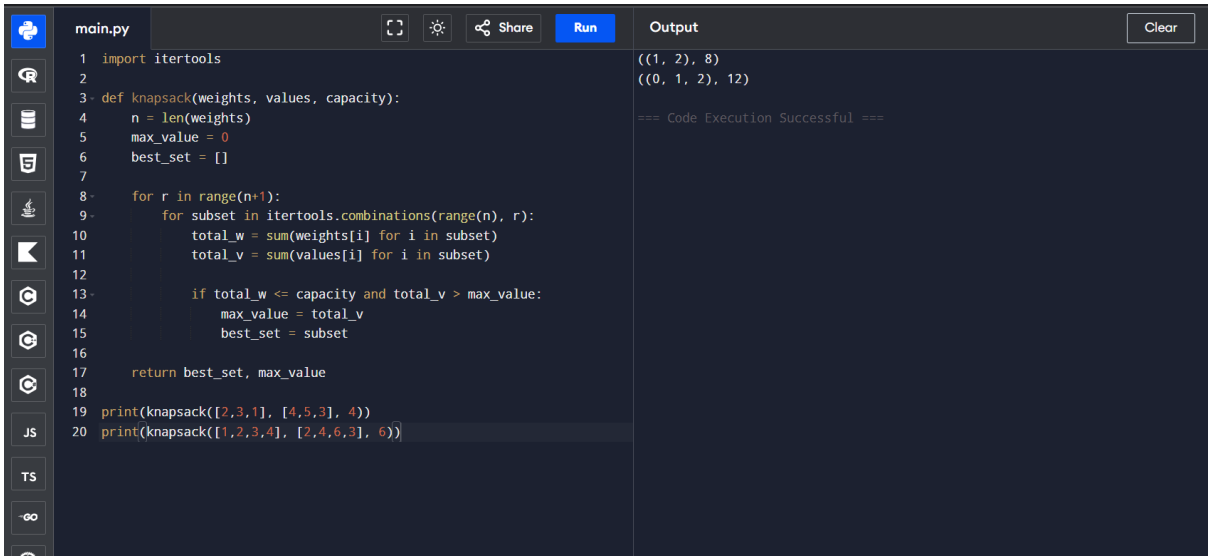
Aim

To solve the 0/1 Knapsack Problem using exhaustive search by finding the subset of items with maximum value without exceeding the given capacity.

Algorithm

1. Start.
2. Read the weights, values, and knapsack capacity.
3. Generate all possible subsets of items.
4. Calculate the total weight of each subset.
5. Check whether the subset is within the given capacity.
6. Calculate the total value of every feasible subset.
7. Compare the values and retain the subset with maximum value.
8. Display the optimal selection and total value.
9. Stop.

Output



Result

Thus, the 0/1 Knapsack Problem was successfully solved using exhaustive search, and the subset having the maximum value within the capacity was obtained.

Time Complexity: $O(2^n \times n)$

Space Complexity: $O(n)$ **auxiliary space.**